

Universidade da Beira Interior

Departamento de Informática



Departamento de
Informática

Interação Humano Com O Computador
— *GenJazz*

Gerador de Progressões Harmónicas de Jazz
Relatório Final

Elaborado por:

Francisco Branco - 51829
Francisco Bondoso - 53905
Afonso Cardoso - 54046

Docente:

Professor Doutor Adriano Raposo

31 de maio de 2026

Agradecimentos

Ao/À docente da unidade curricular, pela disponibilidade e pelos comentários que foram ajudando a afinar as decisões de design ao longo do semestre.

Ao criador da API GenJazz, disponível publicamente, sem a qual grande parte do projeto não seria possível.

À comunidade open-source por trás do React, do Vite e da fonte Inter — ferramentas que acabámos por usar praticamente todos os dias.

Índice

Índice	ii
1 Introdução	1
1.1 Enquadramento e Motivação	1
1.2 Objetivos	1
1.3 Organização do Documento	2
2 Arquitetura do Sistema	3
2.1 Stack e Organização do Código	3
2.2 Gestão de Estado e Autenticação	3
2.3 API GenJazz	4
3 Manual do Utilizador	5
3.1 Acesso e Autenticação	5
3.2 Ecrã Principal e Geração	5
3.3 Resultado, Biblioteca e Perfil	7
4 Decisões de Design e Acessibilidade	8
4.1 Paleta de Cores	8
4.2 Tipografia	8
4.3 Acessibilidade para Utilizadores com Daltonismo	9
4.4 Outras Considerações de Acessibilidade	10
5 Alterações ao Protótipo da 1.^a Parte	12
5.1 Acessibilidade Cromática	12
5.2 Política de Privacidade	12
5.3 Tipografia — Remoção da Playfair Display	13
6 Conclusões e Trabalho Futuro	14
6.1 Conclusões	14
6.2 O Que Não Foi Implementado	14
6.3 Trabalho Futuro	15
A Declaração de Utilização de Inteligência Artificial	16

Acrónimos

IA	Inteligência Artificial
IHC	Interação Humano-Computador
RGPD	Regulamento Geral sobre a Proteção de Dados
SPA	<i>Single Page Application</i>
UBI	Universidade da Beira Interior
UX	<i>User Experience</i>
WCAG	<i>Web Content Accessibility Guidelines</i>

Capítulo

1

Introdução

1.1 Enquadramento e Motivação

Este relatório apresenta a segunda e última fase do projeto **GenJazz**, desenvolvido no âmbito da unidade curricular de Interação Humano-Computador (IHC) do curso de Engenharia Informática da Universidade da Beira Interior (UBI). O GenJazz é uma aplicação móvel que gera progressões harmónicas de jazz a partir de parâmetros configuráveis — tonalidade, estrutura formal e tipo de modulação — e permite ao utilizador ouvir o resultado, visualizá-lo numa grelha de compassos e guardá-lo numa biblioteca pessoal.

A ideia surgiu de um problema concreto: o jazz tem uma linguagem harmónica rica mas pouco acessível a quem está a aprender. Há livros e partituras, claro, mas faltam ferramentas interativas que permitam explorar essa linguagem de forma rápida e auditiva. O GenJazz tenta preencher esse espaço de forma simples e direta.

Na primeira fase foram definidos o conceito, as personas, os fluxos de navegação e o protótipo de baixa-média fidelidade. Nesta fase, o protótipo foi implementado como aplicação web funcional e melhorado com base nas heurísticas de Nielsen ? e em princípios de design inclusivo.

1.2 Objetivos

Os principais objetivos desta fase foram: implementar o protótipo da Parte 1 como aplicação web funcional; integrar a API GenJazz para geração de progressões e conversão para áudio; e adicionar funcionalidades de acessibilidade para utilizadores com deficiência de visão cromática que não tinham sido contempladas no protótipo original.

1.3 Organização do Documento

O documento está organizado da seguinte forma: o Capítulo 2 descreve a arquitetura do sistema; o Capítulo 3 é o manual do utilizador; o Capítulo 4 apresenta as decisões de design e acessibilidade; o Capítulo 5 detalha as alterações face ao protótipo da Parte 1; e o Capítulo 6 contém as conclusões e o que ficou por implementar. O Apêndice A é a declaração de uso de IA.

Capítulo

2

Arquitetura do Sistema

O GenJazz é uma *Single Page Application* ([SPA](#)) construída com **React 18** e **Vite**, que apresenta a interface dentro de uma moldura de smartphone para deixar claro que a experiência foi pensada para mobile. A arquitetura segue um modelo cliente-servidor simples: toda a lógica de apresentação corre no browser do utilizador, e os dados musicais são fornecidos por uma API REST externa alojada em `genjazz-api.fly.dev`.

2.1 Stack e Organização do Código

A escolha do React deveu-se à sua abordagem declarativa e ao sistema de componentes, que tornam mais fácil gerir os vários estados possíveis de uma interface com tantos elementos dinâmicos. O Vite foi preferido ao Create React App simplesmente por ser mais rápido — tempos de arranque e hot-reload visivelmente melhores durante o desenvolvimento.

O código está organizado em `src/screens/` para os ecrãs completos, `src/components/` para componentes reutilizáveis (como o `CircleOfFifths`, o `AudioPlayer` e o `MeasureGrid`), e `src/hooks/` para lógica partilhada.

2.2 Gestão de Estado e Autenticação

O estado global é gerido por dois contextos React. O `AuthContext` trata da autenticação: registo, login e dados do utilizador. As palavras-passe são transformadas em hash SHA-256 via `crypto.subtle` antes de serem guardadas no `localStorage`. A sessão fica no `sessionStorage`, pelo que termina automaticamente quando se fecha o separador.

O `ThemeContext` controla o modo de acessibilidade cromática ativo. Quando o utilizador muda o modo, é adicionado um atributo `data-colorblind` ao elemento `<html>`, o que faz com que as variáveis CSS de acento, erro e su-

cesso sejam substituídas pela paleta correspondente, sem tocar em nenhum componente individualmente.

Toda a lógica da API foi centralizada no hook `useGenJazz`, que trata do carregamento do catálogo, das seleções do utilizador, da geração de progressões, da conversão para MP3 e da biblioteca pessoal.

2.3 API GenJazz

A Tabela 2.1 lista os endpoints utilizados.

Método	Endpoint	Descrição
GET	<code>/api/keys</code> , <code>/api/structures</code> , <code>/api/modulations</code>	Catálogos de opções disponíveis
GET	<code>/api/generate/{k}/{s}/{m}</code>	Gera uma progressão
GET	<code>/api/chords2mp3/{chords}</code>	Converte acordes em MP3
GET/POST/DELETE	<code>/api/chords/{email}/...</code>	Biblioteca pessoal (listar, adicionar, apagar)

Tabela 2.1: Endpoints da API GenJazz.

A biblioteca usa uma estratégia *local-first*: as progressões são guardadas imediatamente no `localStorage` e a sincronização com a API acontece em segundo plano. Assim a aplicação funciona mesmo sem ligação à internet, com os dados locais disponíveis de imediato.

Capítulo

3

Manual do Utilizador

A aplicação funciona em qualquer browser moderno (Chrome 90+, Firefox 88+, Safari 14+), sem instalação. Precisa de ligação à internet para gerar progressões e reproduzir áudio; a biblioteca pessoal fica disponível localmente mesmo sem rede.

3.1 Acesso e Autenticação

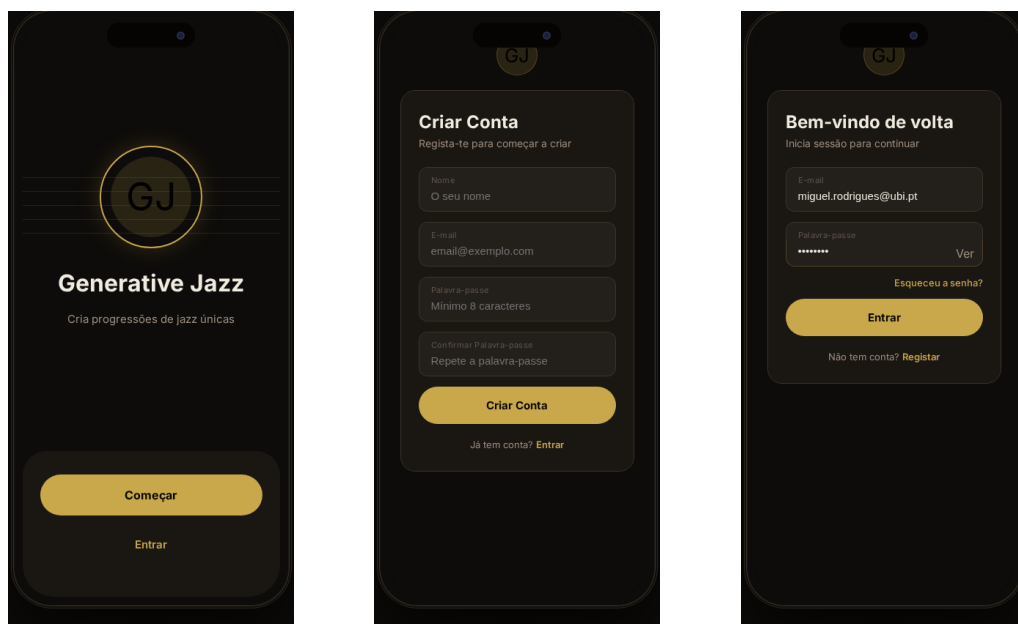
Ao abrir a aplicação aparece o ecrã de boas-vindas (Figura 3.1a) com dois botões: **Começar** para criar conta e **Entrar** para quem já tem. O registo (Figura 3.1b) pede nome, email e palavra-passe com pelo menos 8 caracteres. Os dados ficam guardados localmente no dispositivo — nada é enviado para servidores externos. Para entrar basta o email e a palavra-passe (Figura 3.1c).

3.2 Ecrã Principal e Geração

Depois de entrar, o utilizador chega ao ecrã principal (Figura 3.2a) com uma saudação que muda com a hora do dia, o botão **Gerar** e as progressões mais recentes. Para criar uma nova, toca-se em **Gerar** e configuram-se três parâmetros (Figura 3.2b):

- **Tonalidade** — seleccionada no Círculo das Quintas interativo, ou “Aleatório”;
- **Estrutura** — Random, AABA, AABC ou ABAB;
- **Modulação** — Random, Dominant, Relative, Subdominant, Parallel ou Chromatic.

Depois de tocar em **Gerar Progressão**, uma animação indica que o pedido está a ser processado.

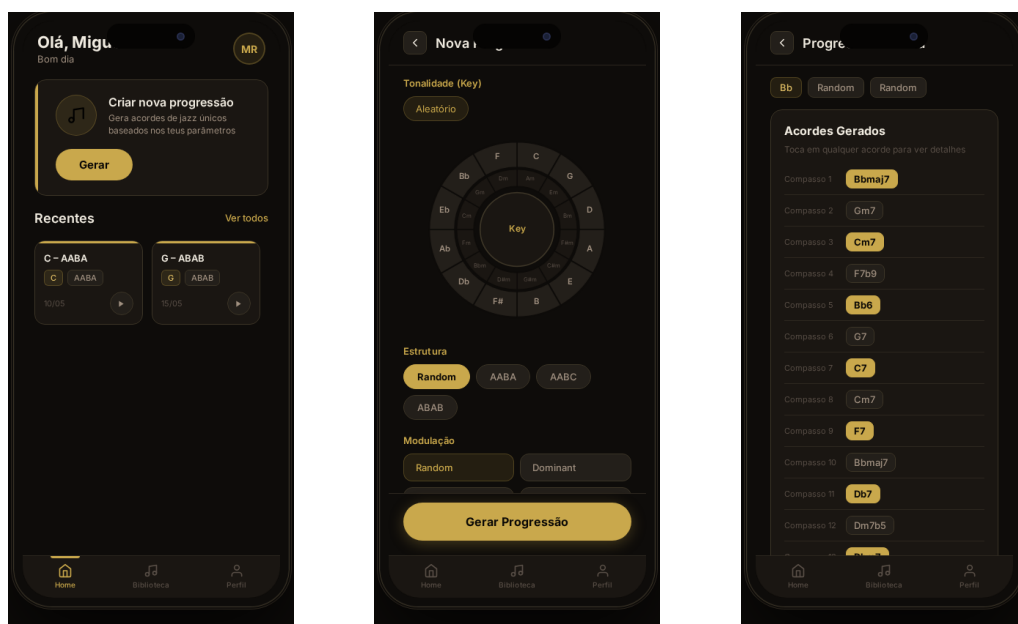


(a) Boas-vindas.

(b) Registo.

(c) Login.

Figura 3.1: Ecrãs de acesso à aplicação.



(a) Ecrã principal.

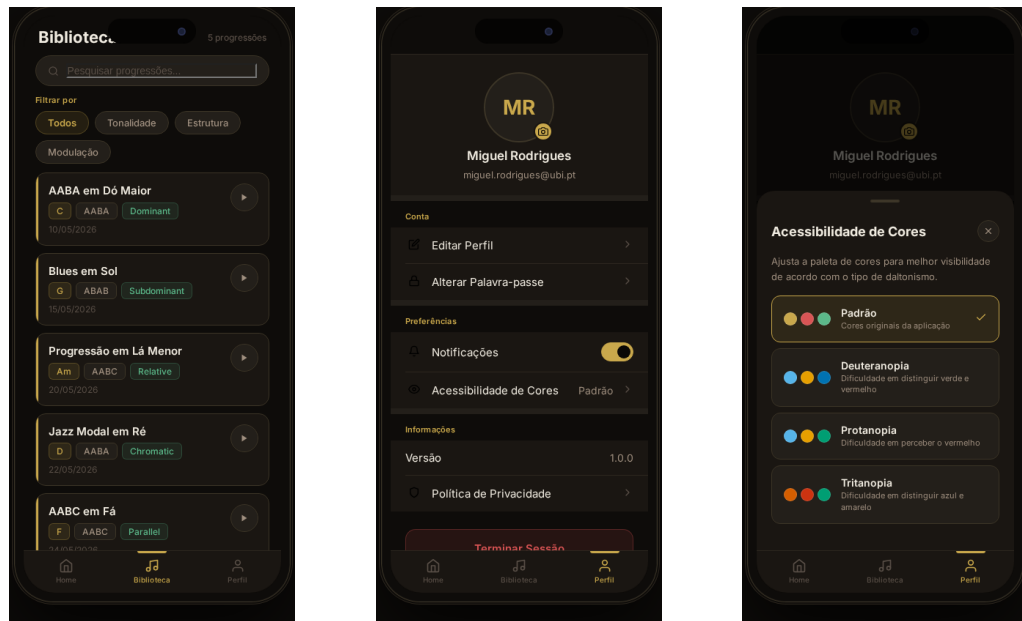
(b) Configuração.

(c) Resultado.

Figura 3.2: Fluxo principal de geração de uma progressão.

3.3 Resultado, Biblioteca e Perfil

O ecrã de resultado (Figura 3.2c) mostra a progressão por compassos, um leitor de áudio com play/pausa e barra de progresso, e o cabeçalho com tonalidade, estrutura e modulação. Durante a reprodução o acorde ativo é destacado em tempo real. Para guardar basta tocar em **Guardar**, editar o nome se necessário, e confirmar.



(a) Biblioteca.

(b) Perfil.

(c) Acessibilidade.

Figura 3.3: Biblioteca, perfil e painel de acessibilidade cromática.

A **Biblioteca** (Figura 3.3a) lista todas as progressões guardadas com pesquisa em tempo real e reprodução rápida em cada item. O **Perfil** (Figura 3.3b) permite editar nome e foto, alterar a palavra-passe, configurar os modos de acessibilidade cromática (Figura 3.3c) e consultar a política de privacidade. A navegação entre ecrãs autenticados é feita pela barra inferior com ícones de casa, nota musical e utilizador.

Capítulo

4

Decisões de Design e Acessibilidade

4.1 Paleta de Cores

A interface usa um tema escuro com um acento dourado/âmbar. Não foi uma escolha por moda — cada cor tem uma razão de ser.

O fundo usa #0e0c0a, um preto com subtom castanho-quente em vez do preto puro. Em ecrãs OLED, o contraste extremo entre fundo preto e texto branco pode ser desconfortável em sessões longas; o subtom quente atenua isso. Visualmente também remete para o ambiente de um clube de jazz, com iluminação ténue, o que se enquadra bem no tema da aplicação.

O acento dourado (#c9a84c) é a cor de todos os elementos interativos. A associação é direta: saxofone, trompete, trombone — os instrumentos centrais do jazz têm exatamente esta cor de latão. Além disso o dourado sobre fundo escuro cumpre os rácios de contraste do *Web Content Accessibility Guidelines* (WCAG) 2.1 nível AA [?], o que não é coincidência. O texto principal usa #f0ebe0, um branco ligeiramente creme, e o secundário #a09080. Ambos mantêm a coerência quente da paleta e criam hierarquia clara sem precisar de tamanhos de letra muito diferentes.

Para estados de erro e sucesso escolhemos deliberadamente um vermelho suave (#d95555) e um verde-azulado (#5cb88a) em vez de verde puro. A razão é simples: verde e vermelho são as cores mais problemáticas para pessoas com daltonismo, e o verde-azulado já cria distância percetiva suficiente para reduzir confusões mesmo sem ativar nenhum modo de acessibilidade.

4.2 Tipografia

Usámos exclusivamente a família **Inter** em pesos de 300 a 700. O protótipo da Parte 1 combinava Playfair Display nos títulos com Inter no corpo, mas

abandonámos essa combinação por três razões práticas.

Primeiro, as serifas da Playfair ficavam com aliasing visível a densidades típicas de smartphones Android. Segundo, dois sistemas tipográficos num espaço compacto criam atrito visual sem benefício real — a hierarquia que a Playfair pretendia introduzir consegue-se perfeitamente com Inter Bold a um tamanho ligeiramente maior. Terceiro, fontes sans-serif com formas abertas são mais fáceis de ler para utilizadores com dislexia ?, e a Inter foi desenhada especificamente para interfaces digitais, com aberturas largas nos caracteres e altura-x elevada.

Elemento	Peso	Tamanho	Uso
Títulos de ecrã	700 Bold	20–22 px	Cabeçalho de cada ecrã
Títulos de secção	600 SemiBold	17–18 px	Títulos de cards e painéis
Corpo	400 Regular	14–15 px	Texto corrido
Labels	500–600	11–13 px	Categorias, metadados
Texto secundário	300–400	11–13 px	Datas, dicas

Tabela 4.1: Hierarquia tipográfica Inter na interface do GenJazz.

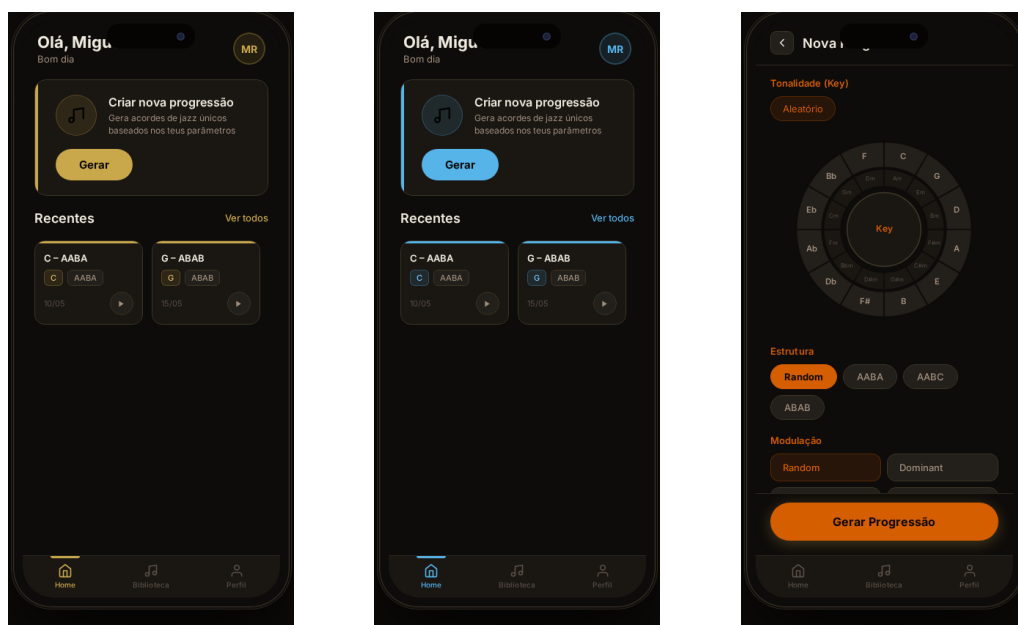
4.3 Acessibilidade para Utilizadores com Daltonismo

A aplicação inclui três modos de acessibilidade cromática, acessíveis nas definições do perfil. As paletas foram baseadas na paleta de Wong (2011) ?, desenvolvida para garantir discriminabilidade nos principais tipos de deficiência de visão cromática. Cada modo substitui apenas as variáveis CSS de acento, erro e sucesso — os fundos e textos mantêm-se inalterados.

Papel	Deuteranopia	Protanopia	Tritanopia
Acento	#56B4E9 azul céu	#56B4E9 azul céu	#D55E00 vermelhão
Erro	#E69F00 laranja	#E69F00 laranja	#CC3311 verm.-laranja
Sucesso	#0072B2 azul escuro	#009E73 verde-azulado	#009E73 verde-azulado

Tabela 4.2: Paletas de acessibilidade para os três tipos de daltonismo suportados.

A deuteranopia — dificuldade em perceber o verde, presente em cerca de 5–6% da população masculina ? — foi o caso que mais justificou esta funcionalidade, uma vez que o dourado da paleta padrão tem uma componente verde que o torna difícil de distinguir neste modo de visão. Para a protanopia (ausência de sensibilidade ao vermelho, ~1%) a paleta é parecida mas com o sucesso ajustado, pois protanopes conseguem distinguir verdes deslocados para o ciano. Para a tritanopia (afeta o azul, ~0,01%) o dourado é substituído por vermelhão, que não depende da distinção azul/amarelo.



(a) Padrão.

(b) Deuteranopia.

(c) Tritanopia.

Figura 4.1: Comparação entre o modo padrão e dois modos de acessibilidade cromática.

A implementação é feita exclusivamente via variáveis CSS: ao mudar o modo, adiciona-se o atributo `data-colorblind` ao elemento `<html>` e as variáveis corretas são automaticamente aplicadas em toda a interface, sem alterar nenhum componente individualmente.

4.4 Outras Considerações de Acessibilidade

Para além dos modos de daltonismo, tomámos algumas decisões que melhoram a acessibilidade geral. Todos os pares de cor texto/fundo foram verificados para cumprir o nível AA do [WCAG 2.1](#). Os botões e elementos tácteis têm pelo menos 44×44 px, seguindo as recomendações da Apple e do

W3C. A hierarquia da interface é transmitida por tamanho, peso e posição, não dependendo só de cor — o que faz com que a interface funcione bem mesmo no modo de alto contraste do sistema operativo.

Capítulo

5

Alterações ao Protótipo da 1.^a Parte

Ao passar do protótipo para a implementação real surgiram três situações em que foi necessário ajustar ou acrescentar funcionalidades. Esta secção descreve o que mudou e porquê.

5.1 Acessibilidade Cromática

A funcionalidade de modos de daltonismo não estava prevista no protótipo original. A necessidade tornou-se óbvia quando percebemos que o dourado da paleta padrão tem uma componente de verde que cria problemas para utilizadores com deuteranopia — o tipo de daltonismo mais comum, que afeta cerca de 5–6% da população masculina ?. Ignorar este problema seria uma barreira de exclusão desnecessária.

A solução foi adicionar três modos alternativos no perfil, cada um com uma paleta baseada nos trabalhos de Wong (2011) ?. O painel de seleção mostra uma pré-visualização das três cores principais antes de o utilizador confirmar, para que possa ver o efeito sem ter de andar a mudar às cegas. A alteração responde diretamente à Heurística 7 de Nielsen — flexibilidade e eficiência — que preconiza que a interface se adapte a diferentes perfis de utilizadores ?.

5.2 Política de Privacidade

Outra adição não prevista foi a secção de Política de Privacidade no perfil. O Regulamento Geral sobre a Proteção de Dados ([RGPD](#)) exige que os utilizadores sejam informados sobre o tratamento dos seus dados mesmo em contexto académico, e mesmo quando tudo fica guardado localmente. Mas há também uma razão de *User Experience* ([UX](#)): um utilizador que

não sabe o que acontece com os seus dados tende a usar a aplicação com desconfiança. A política é curta e direta — explica o âmbito educacional da aplicação, o que a API recebe, e como apagar os dados locais — e isso é suficiente para gerir as expectativas de forma honesta.

5.3 Tipografia — Remoção da Playfair Display

O protótipo usava Playfair Display nos títulos e Inter no corpo. Na implementação passámos a usar apenas Inter em toda a interface. O motivo foi essencialmente prático: ao renderizar em ecrãs de densidade intermédia, as serifas da Playfair tinham aliasing visível que tornava os títulos menos nítidos do que o texto em Inter. Além disso, manter duas famílias tipográficas num ecrã pequeno criava um atrito visual que não justificava qualquer benefício real — a hierarquia ficou igualmente clara usando apenas pesos diferentes da Inter.

Capítulo

6

Conclusões e Trabalho Futuro

6.1 Conclusões

O GenJazz cumpriu o objetivo principal desta fase: transformar o protótipo em algo que funciona de verdade, fiel ao conceito original mas com os ajustes que só a implementação real consegue revelar. O fluxo central — gerar uma progressão, ouvi-la e guardá-la — é rápido e direto, com poucos passos entre o utilizador e o resultado. A arquitetura *local-first* funcionou bem na prática: a interface responde de imediato sem depender da latência da rede para operações básicas.

Do ponto de vista de [IHC](#), a funcionalidade de acessibilidade cromática foi provavelmente a contribuição mais significativa desta fase. Não estava prevista no protótipo e surgiu durante a implementação quando percebemos que a paleta padrão excluía uma parte dos utilizadores sem qualquer necessidade. Resolver isso com paletas baseadas em investigação publicada e uma implementação via variáveis CSS globais foi uma escolha que valeu a pena.

6.2 O Que Não Foi Implementado

Algumas funcionalidades esboçadas na Parte 1 não chegaram a esta versão. A **sincronização de conta entre dispositivos** implicaria um servidor de autenticação próprio que está fora do âmbito deste projeto — a API pública do GenJazz não tem suporte para autenticação de utilizadores. O **modo offline completo com cache de áudio** exigiria um Service Worker com estratégia de cache de ficheiros MP3, o que não foi desenvolvido por falta de tempo. A **exportação de partituras** dependeria de uma biblioteca de renderização musical (como VexFlow) cuja integração teria ultrapassado o âmbito da [IHC](#).

6.3 Trabalho Futuro

As três funcionalidades não implementadas são os candidatos mais naturais para uma versão futura. Para além destas, seria útil fazer uma sessão de testes de usabilidade com utilizadores reais — a avaliação interna tem limites óbvios. Adicionar atributos ARIA adequados para suporte a leitores de ecrã seria também um passo importante para tornar a aplicação verdadeiramente inclusiva.

Apêndice



Declaração de Utilização de Inteligência Artificial

No desenvolvimento deste projeto recorreremos pontualmente a ferramentas de Inteligência Artificial ([IA](#)) generativa em situações específicas.

As ferramentas foram usadas principalmente na **escolha de cores e tipografia**: quando tínhamos dúvidas sobre combinações de cor, rácios de contraste ou sobre qual a fonte mais adequada para determinados contextos de interface, usámos assistentes de [IA](#) para discutir opções e obter referências adicionais. O mesmo se aplicou a **dúvidas estéticas e de propriedades de utilização** ao longo do desenvolvimento — questões sobre espaçamento, hierarquia visual, dimensionamento de elementos tácteis e similares. Foram também usadas para **correção de bugs e erros** que foram surgindo durante o desenvolvimento, como problemas de renderização em determinados browsers ou comportamentos inesperados de componentes React.

As decisões finais — tanto de design como de implementação — foram sempre tomadas pelo grupo. As ferramentas utilizadas foram o **Claude** e o **Gemini**.